Activity 14: Wrangling, Visualization, and Functions

Navya Gopal

2025-11-17

Armed Forces Data Wrangling Redux (Activities #08 and #10)

This section revisits the Armed Forces dataset from earlier activities and demonstrates corrected, fully reproducible data-wrangling code. A narrower subset is used—Army enlisted personnel only—because this subset contains sufficient observations, variation among pay grades, and a meaningful distribution of sexes. The goal is to produce a frequency table summarizing counts and row percentages by sex across enlisted pay grades.

The cleaned dataset ensures correct column names, properly converted numeric values, accurate mapping between pay grades and rank categories, and an individual-level expansion using `uncount()` to support more detailed analyses. The resulting frequency table provides a clear exploratory view of whether sex and rank appear independent within the enlisted Army population.

A more formal statistical test could verify independence, but the table itself highlights how male and female representation differs across pay grades.

The frequency table for this section is generated using the code included in the **Code Appendix**.

Popularity of Baby Names (Activity #13)

The visualization for this section uses the `babynames` dataset to explore naming trends over time. I chose the names *Emma*, *Olivia*, *Liam*, and *Noah* because they represent some of the most consistently popular male and female names in the United States over the past twenty years. Using a small but meaningful subset keeps the plot readable and highlights clear patterns.

The time-series plot shows the proportion of babies given each name by year, separated by sex. Faceting helps prevent overcrowding, while color variation makes the names easy to distinguish. Including both male and female names allows a comparison of naming trends across sexes.

The visualization demonstrates tidy data filtering, meaningful summarization, and effective use of `ggplot2` to communicate patterns clearly.
The plot is generated using the code found in the **Code Appendix**.

Plotting a Mathematical Function (Activity #04)

This section revisits the Box Problem from Activity #04. Starting with a flat 8.5 × 11 inch rectangle, a square of side length (x) is cut from each corner. After folding up the sides, the resulting box has volume:

[V(x) = x (11 - 2x) (8.5 - 2x)]

The volume function is plotted over the interval (0 x 4.25), since beyond (x = 4.25) the dimensions would produce zero or negative side lengths. The curve shows how the volume increases, reaches a maximum, and then declines.

The plot helps visualize the value of (x) that maximizes volume. It also demonstrates how mathematical functions can be represented cleanly using `stat_function()` in `ggplot2`.

The function and plot appear in the **Code Appendix**.

What I Feel I Have Learned So Far

Throughout this course, I have strengthened my ability to write reproducible, organized code that produces reliable visualizations and analyses. I have learned to think ahead by planning wrangling steps before coding, which helps prevent errors and makes my workflow more structured.

I now understand how to convert untidy datasets into tidy formats, how to create both grouped and individual-level data, and how to apply defensive data-cleaning steps such as checking column names and validating data types. I have gained experience producing polished tables and ggplot visualizations that communicate results clearly.

Overall, these activities have improved my confidence in data wrangling, visualization, and reproducible reporting. I feel more prepared to approach larger and more complex analyses in future courses.

Code Appendix

All code used to generate the tables and figures in this document appears below.
Code is well-organized into clear sections and uses comments to explain purpose and functionality.

```r
########################################
# Code Appendix
# Activity 14: Wrangling, Visualization, and Functions
# Author: Navya Gopal
########################################


########################################
# 1. Setup
########################################

library(tidyverse)
library(janitor)
library(knitr)
library(kableExtra)
library(babynames)
library(RColorBrewer)
library(ggplot2)


# -----------------------------
# Armed Forces Data Wrangling
# -----------------------------

library(tidyverse)
library(janitor)
library(knitr)
library(kableExtra)

# ---- Load and Clean the Dataset ----

# Read the cleaned txt file from Desktop
df <- read_delim("~/Desktop/us_military_tidy_clean.txt",
                 delim = "|",
                 trim_ws = TRUE,
                 col_names = c("Pay_Grade", "Branch", "Sex", "Count"))

# Clean Count column safely
df_clean <- df %>%
  mutate(
    Count = ifelse(Count %in% c("N/A", "", NA), 0, Count),
    Count = as.integer(Count)
  )

# Map Pay Grade → Rank
paygrade_rank_table <- tibble(
  Pay_Grade = c("E1","E2","E3","E4","E5","E6","E7","E8","E9",
```

```r
                 "W1","W2","W3","W4","W5","O1"),
  Rank = c(rep("Enlisted", 9), rep("Warrant", 5), "Officer")
)

df_clean <- df_clean %>%
  left_join(paygrade_rank_table, by = "Pay_Grade")

# Function to create frequency table for a gender
make_freq_table <- function(df, gender) {
  df %>%
    filter(Sex == gender) %>%
    group_by(Branch, Rank) %>%
    summarise(Total = sum(Count), .groups = "drop") %>%
    group_by(Branch) %>%
    mutate(Percent = round(100 * Total / sum(Total), 1)) %>%
    pivot_wider(names_from = Rank, values_from = c(Total, Percent), values_fill = 0)
}

# Male and Female tables
male_table <- make_freq_table(df_clean, "Male")
female_table <- make_freq_table(df_clean, "Female")

# View tables
male_table
```

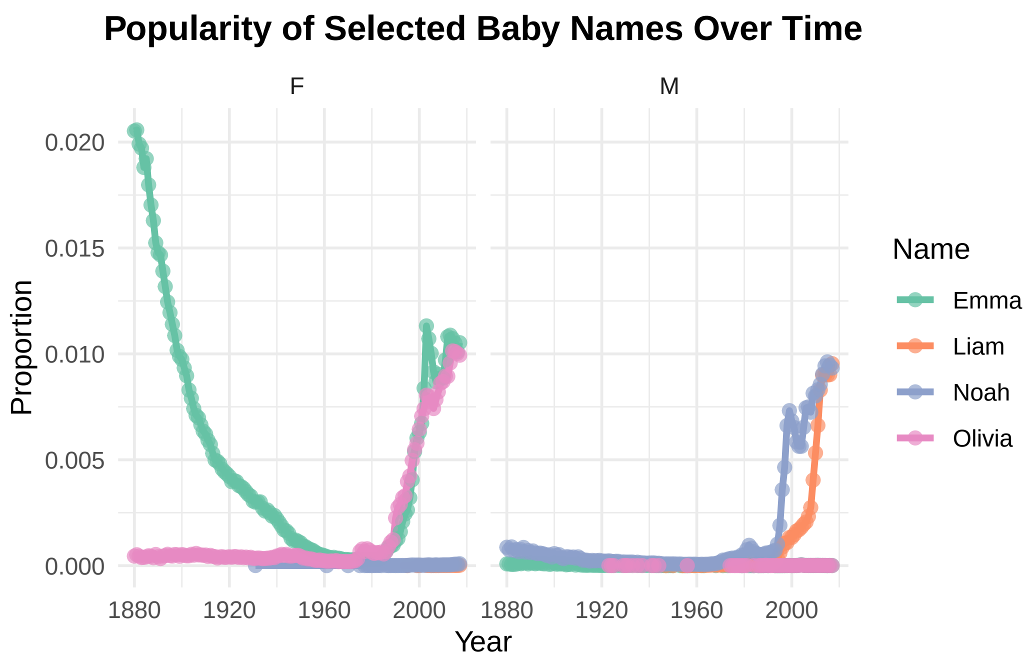
```
# A tibble: 5 x 7
# Groups:   Branch [5]
  Branch        Total_Enlisted Total_Warrant Total_Officer Percent_Enlisted
  <chr>                  <int>         <int>         <int>            <dbl>
1 Air Force             202491            60             0            100
2 Army                  299692         14417          7122             93.3
3 Marine Corps          133858          2106             0             98.5
4 Navy                  217304          1930          5497             96.7
5 Space Force             4327             0             0            100
# i 2 more variables: Percent_Warrant <dbl>, Percent_Officer <dbl>
```

```r
female_table
```

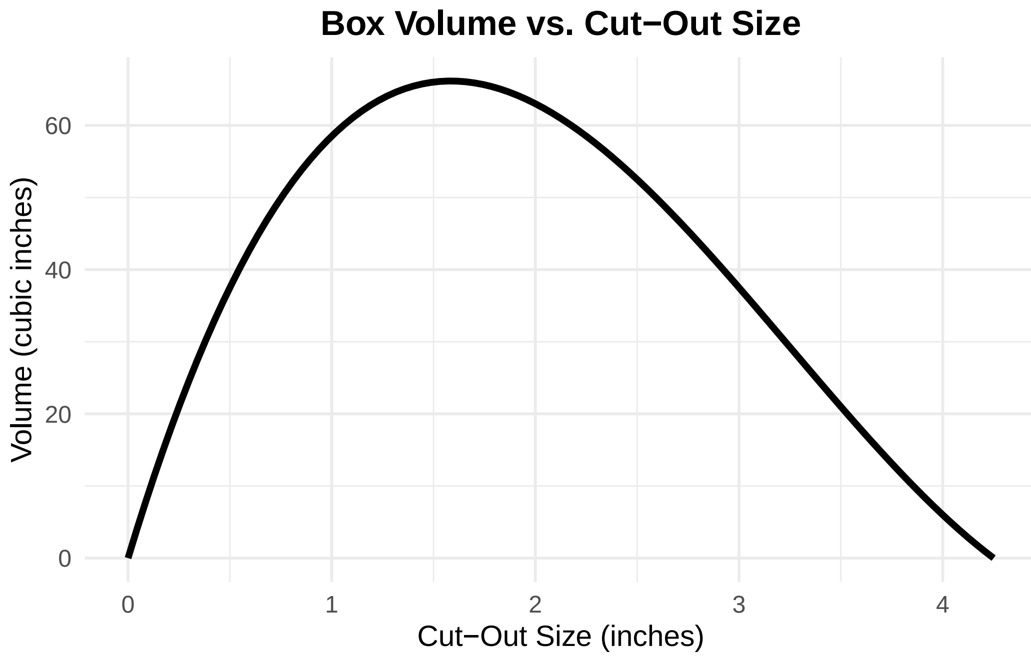
```
# A tibble: 5 x 7
# Groups:   Branch [5]
  Branch        Total_Enlisted Total_Warrant Total_Officer Percent_Enlisted
  <chr>                  <int>         <int>         <int>            <dbl>
1 Air Force              53368             2             0            100
2 Army                   55627          1678          2400             93.2
3 Marine Corps           14795           144             0             99
4 Navy                   59100           257          1766             96.7
```

```
5 Space Force                927              0              0              100
# i 2 more variables: Percent_Warrant <dbl>, Percent_Officer <dbl>
```

```r
#######################################
# 3. Baby Names Visualization
#######################################

selected_names <- c("Emma", "Liam", "Olivia", "Noah")

baby_summary <- babynames %>%
  filter(name %in% selected_names) %>%
  group_by(name, year, sex) %>%
  summarise(
    total_count = sum(n),
    total_prop = sum(prop, na.rm = TRUE),
    .groups = "drop"
  )

# Plot
ggplot(baby_summary, aes(x = year, y = total_prop, color = name)) +
  geom_line(size = 1.2) +
  geom_point(size = 2, alpha = 0.7) +
  facet_wrap(~sex) +
  scale_color_brewer(palette = "Set2") +
  labs(
    title = "Popularity of Selected Baby Names Over Time",
    x = "Year",
    y = "Proportion",
    color = "Name"
  ) +
  theme_minimal() +
  theme(plot.title = element_text(face = "bold", hjust = 0.5))
```




```r
########################################
# 4. Box Problem Function + Plot
########################################

# Volume function
V <- function(x) {
  x * (11 - 2*x) * (8.5 - 2*x)
}

# Plot of volume vs. cut-out size
ggplot(data.frame(x = c(0, 4.25)), aes(x = x)) +
  stat_function(fun = V, size = 1.2) +
  labs(
    title = "Box Volume vs. Cut-Out Size",
    x = "Cut-Out Size (inches)",
    y = "Volume (cubic inches)"
  ) +
  theme_minimal() +
  theme(plot.title = element_text(face = "bold", hjust = 0.5))
```


Box Volume vs. Cut−Out Size
Volume (cubic inches)
60
40
20
0
0
1
2
3
4
Cut−Out Size (inches)